

Solving the sliding puzzle game

Student ID: 862546113

[...] the solution often turns out more beautiful than the puzzle [...]

Richard Dawkins
Unweaving the Rainbow: Science, Delusion and the
Appetite for Wonder

Introduction

People face different problem solving challenges all the time: from our early ancestors searching for food or shelter to modern tasks like planning a route to pick up children from school, stop by the grocery store, and pick up the clothes from the laundry. A large share of tasks people deal with today can be represented as tree search problems. A tree search problem is any problem where decision making can be represented as a tree, which has:

- Start state. This is the initial state, where we start solving the problem.
- Branches. They represented the choices we can make.
- Nodes. The new states to which the choices lead us.
- Target state. The solution.

Although many such tasks are very important and have a direct influence on our lives, there are problems we solve because they seem interesting. One of such problems is a sliding puzzle.

Sliding puzzle game

Sliding puzzle is a task where one needs to arrange numbered tiles on a field by sliding them one by one into adjacent empty space until they are placed in the correct order. The correctness of the order is defined by what the tiles represent: it can be a sequence of numbers or a picture. On Figure 1, there is an example of such a puzzle. On the left, there is a start state. The goal is to move tiles such that the target state is achieved.

Start state				Target state			
1	2		4	1	2	3	4
5	6	3	7	5	6	7	8
9	15	10	11	9	10	11	12
13	14	12	8	13	14	15	

Figure 1: Example of a sliding puzzle on 4x4 grid.

This task can easily be represented as a tree, and getting to the correct order of the tiles is the same as looking for the target position in the tree. Each node in the tree represents a configuration of the tiles, and each branch represents a move made by sliding an adjacent tile into the empty cell. The goal is to find a path from the initial state to the target state.

There are a lot of algorithms for searching a path in such a tree: Depth-First Search, Breadth-First Search, Dijkstra's algorithm, Uniform Search, A*, Bidirectional Search and many others. Each algorithm has its pros and cons. For

algorithms comparison, the following three metrics are commonly used: completeness, optimality, and computational efficiency. Completeness means an algorithm will find the solution if it exists. Some algorithms may not be able to find an existing solution because they get stuck. Optimality is the ability of the algorithm to provide the best solution, which is the shortest path in the tree. We also analyze computational efficiency because time and memory are usually limited.

In this report, the comparison of the following three algorithms run on 3x3 and 4x4 puzzles will be provided: Uniform Search, A* with Manhattan heuristic, and A* with the Misplaced Tiles heuristic.

Algorithms description

1. Uniform Search

When searching the tree, this algorithm chooses the path with the lowest cost, which is the sum of the costs of all moves from the start state to the current state.

2. A* with Manhattan heuristic

A* uses similar idea. Apart from the cost, this algorithm uses heuristic to identify the best move at each time step. A heuristic is an array of rules or ideas that help us to find the best solution. Algorithms also use such ideas. The Manhattan heuristic uses the Manhattan distance as an estimation of whether the choice is good or bad. With this metric, the algorithm estimates how far the tile is located from its target position if the tile was moving only in vertical or horizontal directions. For example, suppose the tile 5 is currently at the position where 1 should be located: first row and first column. Its correct target position in the 3x3 grid is the center cell, which is the second row and the second column. So, its Manhattan distance is $2 - 1 + 2 - 1 = 2$. As a result, the algorithm uses the sum of the cost from the start to the current state and the heuristic.

3. A* with the Misplaced Tiles heuristic

In this algorithm, the heuristic is not the Manhattan distance, but the number of tiles that are not in their target positions.

Experiments

The above three algorithms were compared on several configurations of 3x3 and 4x4 start states. The goal of the experiments was to compare the behavior of the algorithm depending of the solution depth. On each run, the following metrics were calculated: the solution depth, the number of expanded nodes, the maximum number of elements in the queue, and the execution time.

Since not every configuration of tiles lead to the target state, the starting states were chosen with the help of external tools. More specifically, for 3x3 puzzle, I used [1] and for 4x4, I used [2]. Although I could implement the admissibility check for any generated or imputed starting state, I chose a faster way: the states were generated by applying several random valid moves to the target state, which guarantees that the resulting puzzle is solvable. However, the number of random moves is not always equal to the optimal solution depth because some moves can cancel each other. For this reason, there was no possibility to choose the solution depth values. In addition, since several different start states can have the same solution depth, the reported values are averaged over all start states with the same solution depth. Therefore, on figures below, "average expanded nodes" means the average number of expanded nodes for all test cases with a specific solution depth, and "average maximum queue size" means the average value of the maximum queue size during in these tests.

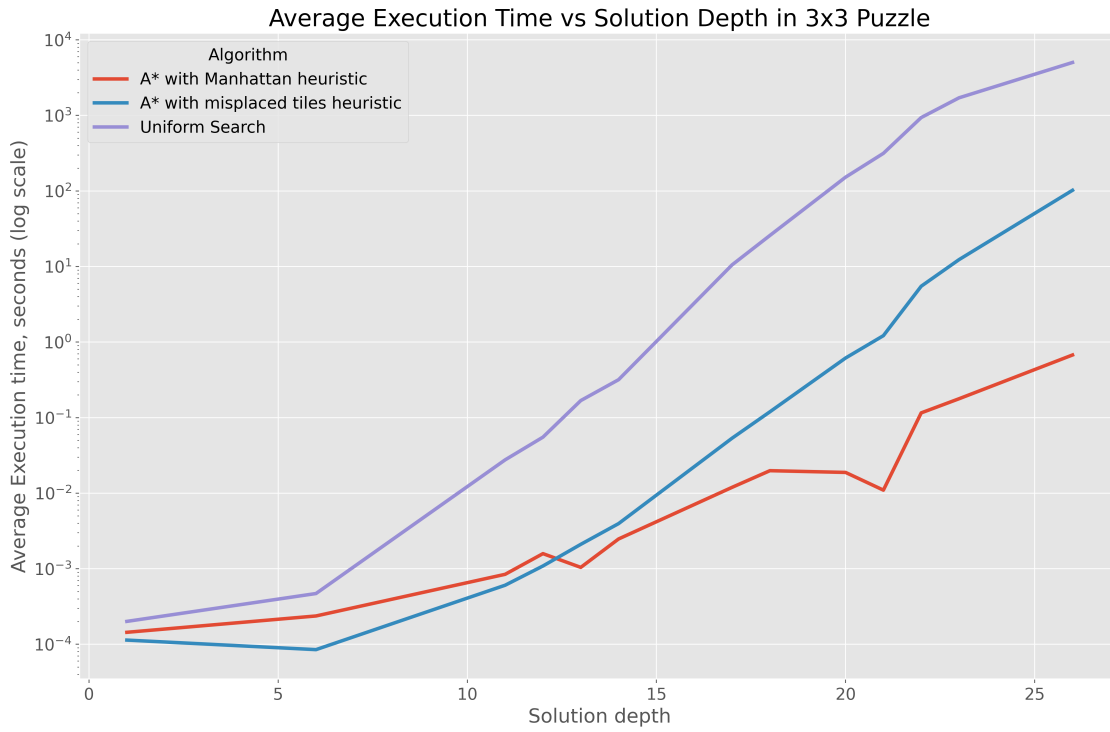


Figure 2: Comparison of the average execution time and solution depth for 3x3 puzzle.

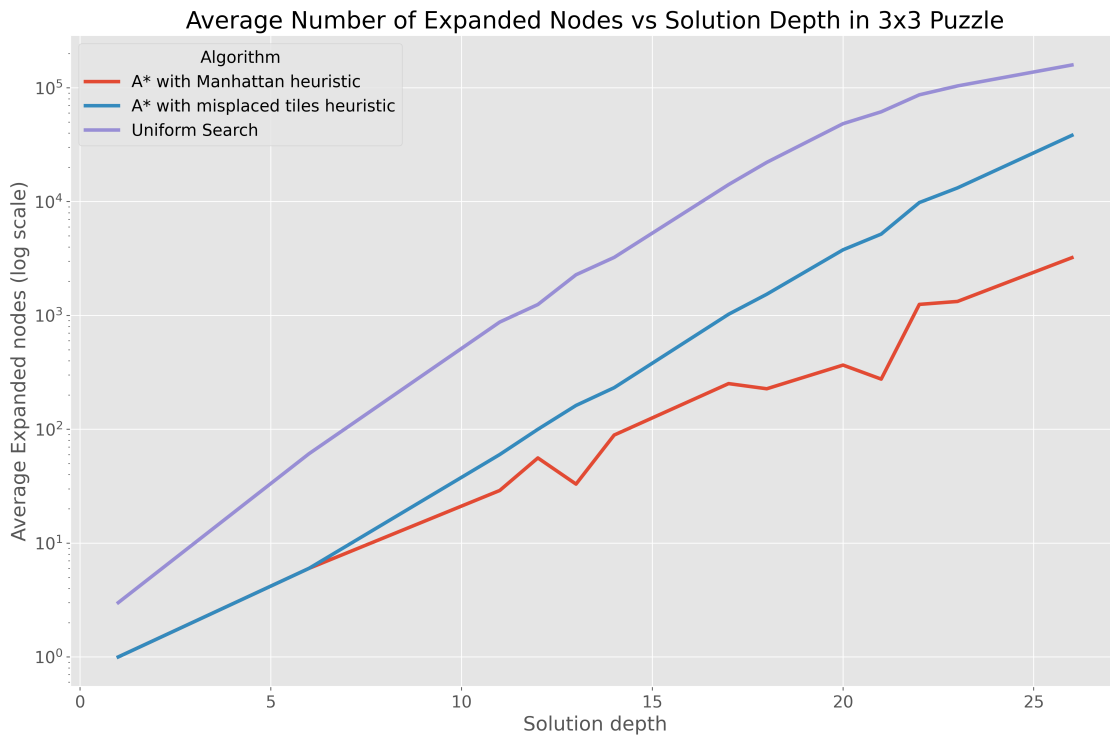


Figure 3: Comparison of average number of expanded nodes and solution depth for 3x3 puzzle.

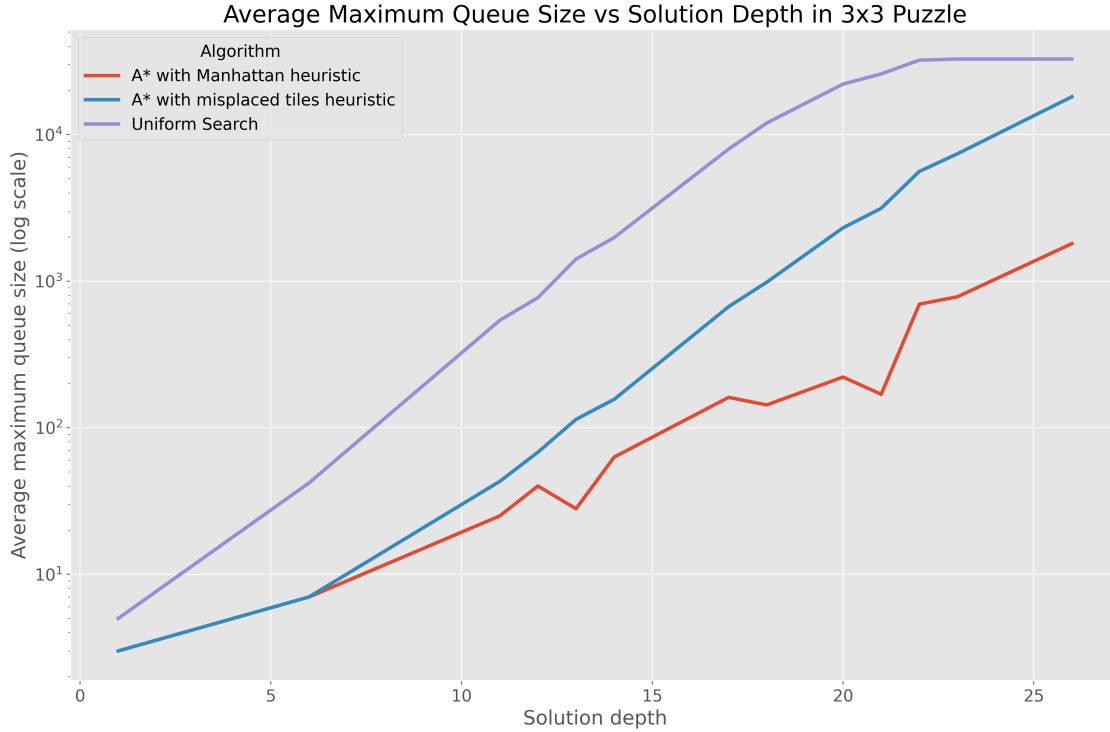


Figure 4: Comparison of average maximum queue size and solution depth for 3x3 puzzle.

Figure 2 shows how the average execution time of the three algorithms changes as their solution depths increase. The result is represented in the logarithmic scale because the execution time values differ significantly. The Uniform Search is the slowest algorithm, which is especially noticeable on large depth values. Both A* algorithms are faster, and A* with Manhattan heuristic is the most efficient one. This is because A* algorithms have additional knowledge about the current state in comparison to the Uniform Search. Uniform search, on the contrary, relies only on the cost value and expands nodes level by level.

Figure 3 shows how the average number of expanded nodes changes as the depth grows. Uniform Search has the largest number of expanded nodes on average, because it searches for the target state level by level. Both A* expand less nodes on average, while A* with Manhattan distance is the most efficient in this comparison.

Figure 4 shows that, on average, the maximum queue size value is largest for the Uniform Search and lowest for the Manhattan A*.

As a result, in 3x3 puzzle experiment, A* with Manhattan heuristic had the smallest averaged maximum queue size, expanded nodes, and required the lowest amount of time to find the target state. Manhattan A* is the most space- and memory-efficient algorithm in these experiments.

For the 4x4 puzzle, Uniform Search was not used on versions where the solution depth is large because it was too slow and could not finish in 6 hours.

Solution depth	Expanded nodes	Max queue size	Time (sec)
5	71	79	0.000771
10	3305	3233	0.386834
15	110621	102531	1297.388588

Table 1: Uniform Search performance for the 3 15-puzzle tests

The results in Table 1 show that Uniform Search is not the best choice when the depth of the solution is large. At depth 15, the search becomes significantly more expensive in comparison to the other 2 depth values: it expands 110621 nodes, the maximum queue size grows to 102531, and the time increases to more than 1200 seconds. For this reason, further experiments were not completed for Uniform Search, and the following comparison focuses more on the two A* algorithms.

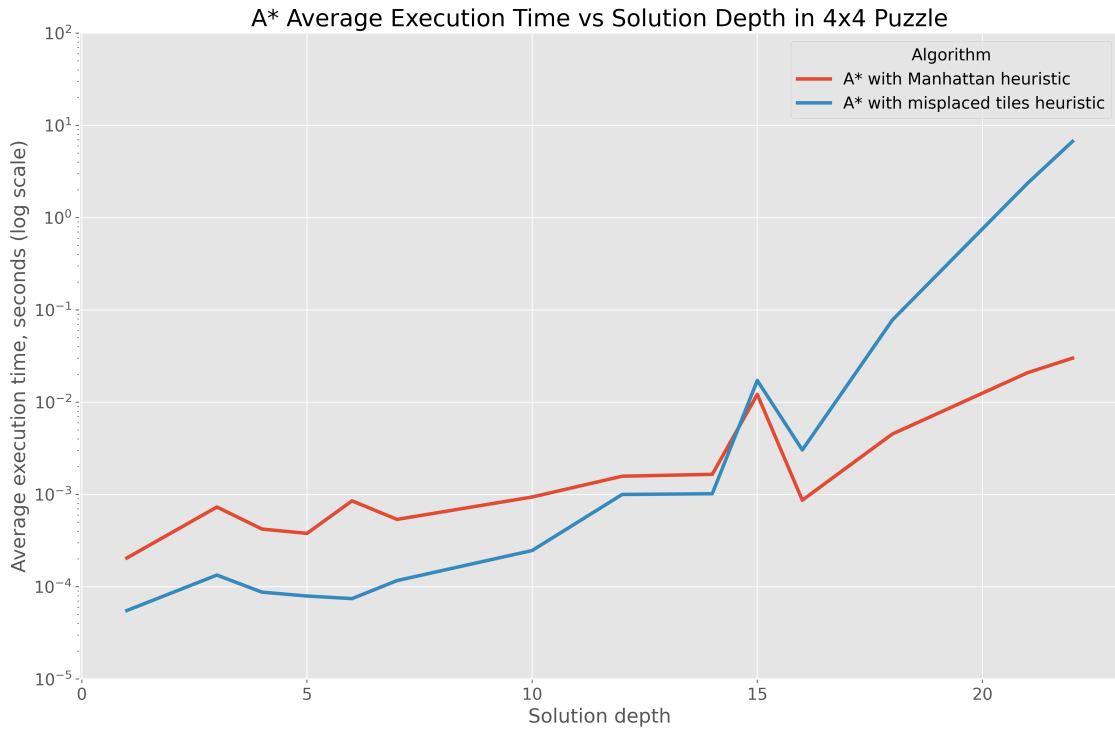


Figure 5: Comparison of the average A* execution time and solution depth for 4x4 puzzle.

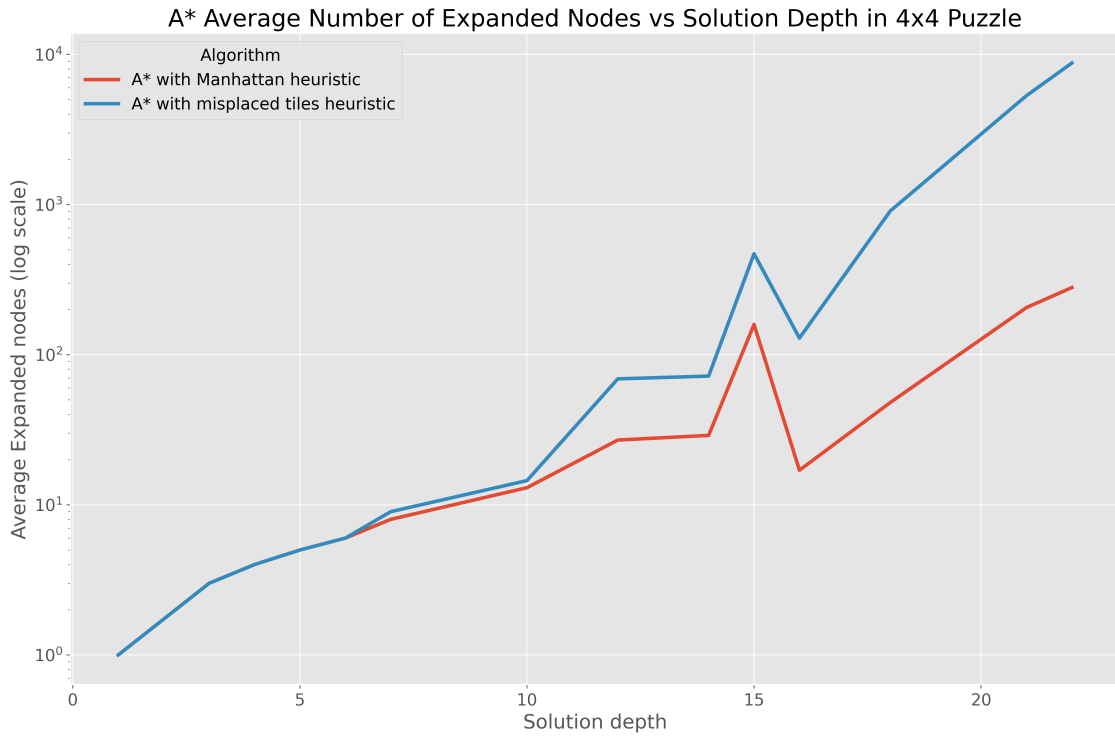


Figure 6: Comparison of average A* number of expanded nodes and solution depth for 4x4 puzzle.

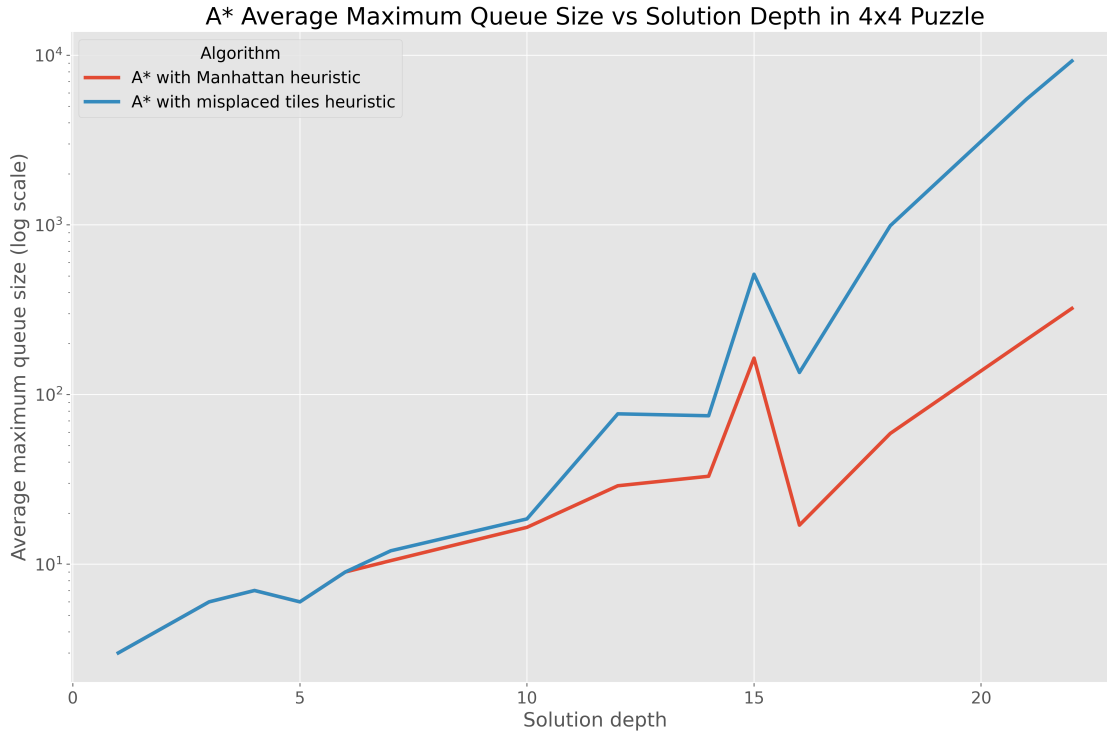


Figure 7: Comparison of average A* maximum queue size and solution depth for 4x4 puzzle.

Figure 5 shows the average execution time of the two A* algorithms for the 4x4 puzzle. For small depth values, both algorithms work similarly. As depth increases, the A* with the Manhattan heuristic becomes faster.

On figure 6, we can see how the average number of expanded nodes changes for the two A* algorithms. Both algorithms behave similarly, but when the solution depth is larger than 15, the difference becomes more noticeable. In this comparison, the Manhattan heuristic also seems a better choice.

From figure 7, we can see similar result for the average maximum queue size. Obviously, the maximum queue size grows as the solution depth increases. As in the previous result, for deeper cases, the Manhattan heuristic is more preferable.

As a result, both A* algorithms perform better than the Uniform Search, and the Manhattan distance is a better estimate than the Misplaced Tiles heuristic, as it gives an algorithm more useful information about the current state.

Conclusion

The results of the experiments are similar for 3x3 and 4x4 cases. Uniform Search is comparable to both A* algorithms in terms of efficiency only if the solution depth is small. When the solution depth is large, this algorithm becomes very expensive and may require hours to find the solution of the depth 20. Both A*, on the contrary, perform much better. They both require less time and memory. However, the choice of heuristic has a noticeable influence on the performance. The results show that the Manhattan is more effective, as algorithm with such a heuristic requires less time to find the solution, uses less memory to store data, and expands less nodes. Perhaps the reason for this is this heuristic gives more geometric information about the tiles, while the Misplaced Tiles heuristic tells only the number of incorrectly positioned tiles. Moreover, Manhattan distance of one tile is always more or equal to the Misplaced Tiles value for this tile. Since both heuristics are admissible, the Manhattan distance is at least not worse.

References

- [1] Sliding Toys. *8 puzzle Sliding Toys*. <https://sliding.toys/mystic-square/8-puzzle/>
- [2] Netlify. *15 Puzzle*. <https://15puzzle.netlify.app/>
- [3] Lecture slides.

- [4] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach, 4th Edition*. Pearson.
- [5] *Setting the recursion limit*. <https://www.geeksforgeeks.org/python/how-to-fix-recursionerror-in-python/>
- [6] Overleaf. *LaTeX documentation*. <https://www.overleaf.com/learn>
- [7] *Heap queue in Python*. <https://docs.python.org/3/library/heapq.html>
- [8] GeeksforGeeks. *Heap and Priority Queue using heapq module in Python*. <https://www.geeksforgeeks.org/python/heap-and-priority-queue-using-heapq-module-in-python/>